

AI Voice Assistant Backend

Detailed Codex-agent implementation guide for an Exotel + Spring Boot + WebSocket + STT + LLM + TTS prototype.

Goal: Build a working inbound phone AI assistant for farmers. A caller speaks Hindi/Hinglish, the backend understands the request, optionally calls a controlled market-data tool, and sends a spoken answer back.

SECURITY WARNING: The Exotel API key and API token were pasted into the chat. Treat them as exposed. Rotate/revoke them in Exotel before using the integration. This guide deliberately does not reproduce the secrets. Never put credentials in Codex prompts, AGENTS.md, screenshots, source code, or Git history.

Build order: repository inspection -> Spring Boot health -> WebSocket -> Exotel protocol -> public WSS -> STT/TTS/LLM interfaces -> PostgreSQL market tools -> end-to-end call -> security and latency hardening.

1. Best Codex Model and Intensity

Recommended: GPT-5.6 Sol. OpenAI currently describes GPT-5.6 Sol as its flagship model for complex professional work, reasoning, and coding. Source: OpenAI Model Guidance.

For your Plus plan: use **GPT-5.6 Sol + High** reasoning for the main implementation. OpenAI's current ChatGPT documentation says Plus includes Medium and High; Extra High and Pro are not included in Plus. Source: OpenAI Help Center, GPT-5.6 in ChatGPT.

Use Medium for small edits, formatting, simple tests, and routine fixes to reduce usage. Use **High** for architecture, WebSocket/audio work, external integrations, debugging, and security reviews.

Important: Codex usage depends on model, task complexity, context, reasoning, tools, and runtime. Large codebase tasks consume more than small tasks. Keep the agent focused on one phase at a time.

2. Master Prompt for Codex

You are the senior backend engineer for an AI phone voice assistant for an agricultural application.

PRIMARY GOAL

Build an inbound Exotel AgentStream voice assistant using Java 21 + Spring Boot. The caller speaks Hindi/Hinglish. The server

STACK

- Java 21
- Spring Boot 3.x
- Maven
- Spring Web + Spring WebSocket
- PostgreSQL
- Docker Compose for local database
- Provider interfaces for STT, TTS, and LLM so providers can be swapped
- Exotel AgentStream VoiceBot Applet / bidirectional WebSocket

SECURITY

- Never hard-code credentials.
- Use environment variables and .env.example placeholders.
- Never log API keys, tokens, Authorization headers, or sensitive caller data.
- Never give the LLM direct database access.
- LLM can request only allow-listed backend tools.
- V1 tools are READ-ONLY.
- Do not implement offer acceptance, transaction creation, cancellation, payment, or other state-changing actions yet.
- Validate every tool parameter server-side.
- Never commit .env.

EXOTEL PROTOCOL

Support connected, start, media, dtmf, stop, mark, and clear events.

Exotel media is base64-encoded raw/slin 16-bit PCM, mono. Support configurable 8000/16000/24000 Hz and verify the configured

Use the current official Exotel AgentStream documentation as the protocol authority. Do not invent undocumented behavior.

ARCHITECTURE

Exotel -> WSS -> VoiceSession -> AudioDecoder -> STT -> ConversationOrchestrator -> LLM -> ToolRouter -> MarketService/PostgreSQL

V1 JOURNEY

Caller connects.

Bot initializes session.

Bot gives a short Hindi/Hinglish greeting.

Caller: "Aaj pyaaz ka kya bhaav hai?"

STT -> text.
LLM -> intent and tool request.
Tool: get_market_price.
Backend validates request and queries PostgreSQL.
LLM -> concise answer.
TTS -> PCM.
Server -> Exotel.
Caller hears the answer.

TOOLS

```
get_market_price(crop, market, date)
get_price_history(crop, market, days)
get_nearby_markets(location, crop)
```

DATA MODEL

```
Farmer(id, name, phone, language, location)
Market(id, name, latitude, longitude)
Crop(id, name)
MarketPrice(id, crop_id, market_id, price_date, min_price, max_price, modal_price, source, source_timestamp)
```

PROJECT MODULES

```
voice/
ai/
market/
farmer/
config/
security/
common/
```

IMPLEMENTATION RULES

1. Inspect the existing repository before modifying it.
2. Preserve existing architecture and conventions.
3. Use a modular monolith for the prototype; do not introduce microservices without a concrete need.
4. Use DTOs for external protocol payloads.
5. Use structured logs correlated by callSid and streamSid.
6. Add unit/integration tests at every phase.
7. Keep provider integrations behind interfaces.
8. Use bounded queues and timeouts around external AI services.
9. Keep V1 simple and deterministic.
10. Do not proceed past a blocking test failure silently.

DEFINITION OF DONE

- GET /health works.
- WebSocket /media accepts a connection.
- Exotel connected/start/media/dtmf/stop/mark/clear events are parsed safely.
- Media is decoded to PCM safely.
- STT adapter works.
- LLM tool calling is allow-listed.
- Market tools return PostgreSQL data.
- TTS adapter produces compatible audio.
- Outbound audio is chunked correctly.
- Real inbound Exotel call reaches the server and receives an AI response.
- Secrets are externalized.
- Tests pass.
- README contains setup, environment variables, Exotel configuration, tunnel/deployment, testing, and troubleshooting.

PHASES

- 0: inspect repo + AGENTS.md
- 1: Spring Boot foundation + health
- 2: WebSocket + Exotel protocol parser + tests
- 3: public WSS
- 4: STT/TTS/LLM interfaces and adapters
- 5: PostgreSQL + seeded market data + tools
- 6: conversation orchestration
- 7: Exotel end-to-end
- 8: latency/security/observability/hardening

AFTER EACH PHASE

- run tests
- report exact commands
- report changed files
- report failures honestly
- update README
- stop if a blocking issue exists
- propose the next phase

3. Phase 0 - Inspect and Initialize

Open the actual repository in Codex. First ask the agent to inspect the tree, build files, Java version, Spring Boot version, database setup, authentication, existing AI code, and environment configuration. Do not let it rewrite the project.

If AGENTS.md is missing, use Codex *!init* to generate the project instruction scaffold. Then review it before proceeding.

Prompt: "Inspect the repository only. Do not modify application code. Identify the existing backend architecture, Java/Spring versions, database setup, AI integrations, and environment handling. Propose the smallest safe plan for adding the voice assistant module."

4. Phase 1 - Spring Boot Foundation

Create or extend the backend with a health endpoint and the minimum WebSocket dependencies. Do not add unnecessary infrastructure.

```
GET /health
Response:
Voice Assistant Server is running
```

```
Run:
./mvnw test
./mvnw spring-boot:run
```

```
Windows:
.\mvnw.cmd test
.\mvnw.cmd spring-boot:run
```

Definition of done: application starts on port 8080, /health returns HTTP 200, and the health test passes.

5. Phase 2 - Exotel WebSocket

Create a WebSocket endpoint such as */media*. Implement a DTO/dispatcher model instead of putting every event in one giant handler.

```
WebSocket /media

connected -> create session
start      -> store callSid, streamSid, caller, media format
media      -> base64 decode -> PCM pipeline
dtmf       -> optional fallback
mark       -> playback correlation
clear      -> clear buffered bot audio/context when needed
stop       -> close session and release resources
```

Exotel's current VoiceBot documentation specifies these events and bidirectional audio. Its documented media payload is base64-encoded raw/slin 16-bit PCM, mono. It supports 8, 16 and 24 kHz; Exotel recommends 16 kHz for most voicebot integrations. Official source: Exotel AgentStream VoiceBot Applet.

Add tests for every event, malformed JSON, missing fields, invalid base64, wrong audio format, duplicate lifecycle events, and disconnect handling.

6. Phase 3 - Public WSS

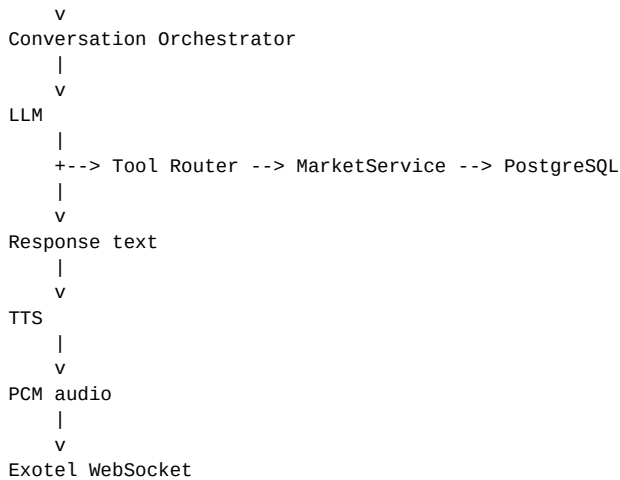
Exotel cannot connect to localhost. Use a development tunnel or deploy the service to a server with a valid TLS certificate. The final endpoint should look like **wss://your-domain.example/media**.

Exotel's VoiceBot Applet accepts a WSS endpoint and also documents a dynamic HTTPS resolver that returns a WSS URL. Keep any tunnel/cloud credentials outside Git.

Test the public WSS with a WebSocket client before connecting Exotel.

7. Phase 4 - STT, LLM and TTS

```
Incoming audio
|
v
STT
|
```



Keep provider interfaces separate: `SpeechToTextService`, `LargeLanguageModelService`, `TextToSpeechService`. This prevents a provider change from forcing a rewrite of the telephony layer.

Exotel is the media/telephony bridge; your bot server provides transcription, NLU/LLM logic, tools and TTS. Official Exotel extension documentation explicitly states that Exotel does not provide built-in STT/TTS for this architecture.

8. Phase 5 - PostgreSQL and Market Tools

Use seeded data first. Do not wait for a live government-data integration to prove the voice loop.

```

Crop
Market
MarketPrice
Farmer

```

MarketPrice fields:

```

id
crop_id
market_id
price_date
min_price
max_price
modal_price
source
source_timestamp

```

First tool: **get_market_price**. Return structured data including price date, modal price, min/max, source, and freshness. The LLM must never invent a price when the backend returns no data.

Then add `get_price_history` and `get_nearby_markets`.

9. Phase 6 - Controlled LLM Tool Calling

```

Farmer:
"Aaj pyaaz ka kya bhaav hai?"

STT
-> text

LLM
-> tool request:
    get_market_price(crop=onion, market=nearby, date=today)

Backend
-> validate
-> query PostgreSQL
-> structured result

LLM
-> concise Hindi/Hinglish answer

```

Critical rule: AI understands; backend verifies and executes. Never allow the model to issue arbitrary SQL or directly mutate database state.

For V1, only read-only tools are enabled. Transactional tools will be a later phase and must require explicit confirmation plus authentication.

10. Phase 7 - Hindi/Hinglish Conversation

Test with: "Aaj pyaaz ka kya bhaav hai?", "Indore mein pyaaz ka rate kya hai?", "Kal ka bhaav kya tha?", and "Mere paas kaunsa market accha hai?".

When crop, market, date, or location is ambiguous, ask a short clarification instead of guessing. Keep responses short because phone calls are latency-sensitive.

Add fallback behavior for STT uncertainty and DTMF if speech recognition fails.

11. Phase 8 - TTS and Audio Playback

Convert TTS output into the exact audio format configured for the Exotel stream. Current Exotel documentation specifies 16-bit linear PCM, mono, with 8/16/24 kHz options.

Send audio in small chunks. Exotel documents a minimum of roughly 100 ms and advises chunk sizes that are multiples of 320 bytes. Avoid giant audio frames.

Implement cancellation/clear handling later for barge-in. First make the simple request -> response loop stable.

12. Latency and Reliability

Measure each stage independently: STT first result, LLM first token, tool latency, TTS first audio, and total time to first response audio.

Use explicit timeouts and graceful fallbacks. Exotel's current AgentStream extension guide warns that if the bot server does not respond within 10 seconds, the session can fail.

Do not make the LLM perform unnecessary reasoning during ordinary phone turns. For the actual runtime voice model, optimize for low latency; the Codex coding model and the production voice LLM are separate choices.

13. Environment and Secrets

```
.env.example
SERVER_PORT=8080

EXOTEL_ACCOUNT_SID=<account-sid>
EXOTEL_API_KEY=<rotated-api-key>
EXOTEL_API_TOKEN=<rotated-api-token>
EXOTEL_PHONE_NUMBER=<exophone>

DATABASE_URL=jdbc:postgresql://localhost:5432/voice_assistant
DATABASE_USERNAME=<local-user>
DATABASE_PASSWORD=<local-password>

LLM_API_KEY=<provider-key>
LLM_MODEL=<model>

STT_API_KEY=<provider-key>
TTS_API_KEY=<provider-key>
```

Never paste the real values into AGENTS.md or a Codex prompt. Use local environment variables or a secure secret store. Add .env to .gitignore.

14. Security Checklist

Area	Requirement
Exotel secrets	Rotate the credentials exposed in chat; use new credentials.
Source control	Never commit .env or secrets.
Logs	Redact tokens, Authorization headers and sensitive caller data.
LLM	No direct DB access.
Tools	Allow-list and validate every parameter.

Transactions	V1 read-only. Later writes need confirmation + authentication.
Caller auth	Phone number alone must not authorize sensitive actions.
WSS	Use TLS and supported Exotel authentication/IP controls.
Audio/data	Define retention before production.

15. Testing and Definition of Done

1. /health -> 200
2. /media WebSocket handshake succeeds
3. connected -> session created
4. start -> callSid/streamSid stored
5. media -> PCM decoded
6. STT -> text
7. LLM -> approved tool request
8. Tool -> market data
9. LLM -> concise response
10. TTS -> PCM
11. PCM -> Exotel media frames
12. Caller hears response
13. stop -> resources released
14. Unit/integration tests -> green
15. No secrets in repository

Ask Codex to run the relevant Maven tests after every phase and report exact commands and failures. Never accept "looks good" without a test or manual verification where applicable.

16. Exotel Configuration After AgentStream Access

Once Exotel enables AgentStream for your account, configure a VoiceBot Applet in the call flow and point it to your public WSS endpoint. Exotel's official documentation covers the WSS endpoint, optional authentication, sample-rate configuration, custom parameters, and bidirectional media.

Keep the first flow simple: incoming call -> VoiceBot Applet -> next step/passthru as required. Do not add complicated IVR routing until the AI loop is working.

If Exotel asks for streaming access, provide the Account SID and use case through official support/compliance channels. Never send API tokens to support unless Exotel's secure credential workflow explicitly requires it.

17. Codex Phase Prompts

Phase 1

Inspect the repository and implement only the health endpoint and voice-module skeleton. Run tests. Do not implement AI.

Phase 2

Implement the Exotel WebSocket protocol with connected/start/media/dtmf/stop/mark/clear, DTOs, lifecycle handling, PCM decoding and tests.

Phase 3

Make /media reachable over WSS for development. Document the tunnel/deployment setup and verify the public WebSocket.

Phase 4

Add STT, LLM and TTS provider interfaces plus adapters. Add timeouts, error handling and tests.

Phase 5

Implement PostgreSQL entities, repositories, seed data and the three read-only market tools.

Phase 6

Implement the conversation orchestrator and allow-listed tool calling. Add Hindi/Hinglish prompts and ambiguity handling.

Phase 7

Connect Exotel -> STT -> LLM -> tool -> TTS -> Exotel and run a real inbound call.

Phase 8

Perform a security, latency, observability and reliability pass. Run the full test suite and update README.

18. 3-4 Day Execution Plan

Day	Target
1	Repository inspection, AGENTS.md, Spring Boot module, health endpoint, WebSocket protocol, tests.
2	Public WSS, Exotel integration if enabled, real audio, STT/TTS, session lifecycle.
3	LLM orchestration, PostgreSQL seed data, market-price tool, Hindi/Hinglish, end-to-end call.
4	Latency, security, error handling, tests, observability, deployment and demo cleanup.

19. Important: What to Do With the Credentials You Just Sent

Because the API key and token were exposed in the conversation, **rotate/revoke them now**. After rotation, create a local .env file and put the new values there. The PDF intentionally contains only placeholders.

Your ExoPhone number is not a secret in the same sense as the API token, but still avoid publishing it unnecessarily. Treat the API key/token as compromised until rotated.

20. Official References

OpenAI Model Guidance: <https://developers.openai.com/api/docs/guides/latest-model>

OpenAI GPT-5.6 in ChatGPT / Codex: <https://help.openai.com/en/articles/20001354-gpt-56-in-chatgpt/>

OpenAI Using Codex with your ChatGPT plan: <https://help.openai.com/en/articles/11369540/>

Exotel VoiceBot Applet / AgentStream: <https://docs.exotel.com/exotel-agentstream/voicebot-applet>

Exotel AgentStream extension guide:

<https://docs.exotel.com/exotel-agentstream/updated-extension-guide-working-with-the-stream-and-voicebot-applet>